

ASP.NET Core Web API AppSettings.json file

Back to: [ASP.NET Core Web API Tutorials](#)

AppSettings.json in ASP.NET Core Web API

Configuration is a critical part of any application because it allows developers to separate environment-specific or changeable values from the core codebase. In ASP.NET Core Web API, configuration is handled in a flexible and structured manner through multiple sources, including JSON files, environment variables, and command-line arguments.

The center is located in the **AppSettings.json** file, which is the default configuration file used to store key-value pairs for application settings. Understanding how this file works, along with environment-specific variants, is essential for building scalable and maintainable applications.

Different Configuration Sources Available in ASP.NET Core Web API

ASP.NET Core allows configuration to be loaded from multiple sources. This flexibility ensures that applications can adapt to different environments and deployment scenarios. Some of the key configuration sources include:

- **AppSettings.json File** → Default configuration file with key-value pairs.
- **Environment-Specific JSON Files** → e.g., appsettings.Development.json, appsettings.Production.json.
- **User Secrets** → For sensitive data like API keys in development.
- **Environment Variables** → Read from OS-level variables.
- **Command-Line Arguments** → Pass values at runtime.
- **Azure Key Vault / External Providers** → For cloud applications requiring secure storage.

The system merges these sources, and in case of conflicts, the later source overrides the earlier one.

What is the ASP.NET Core AppSettings.json File?

The **AppSettings.json** file is a JSON-formatted file in ASP.NET Core that stores application configuration data. It serves as a central repository for storing application-specific settings, such as connection strings and logging details, in a key-value format. In real-time applications, it may include:

- **Database Connection Strings**
- **Logging Configurations**
- **API Keys or Third-Party Service Credentials**
- **Email Server and SMS Settings**
- **Application-Specific Settings** (e.g., Pagination Size, Caching Duration)

It serves as a **centralized place** to store non-sensitive and environment-agnostic configuration values.

Environment-Specific AppSettings.json Files

ASP.NET Core introduces environment-specific configuration files, such as:

- appsettings.Development.json
- appsettings.Staging.json
- appsettings.Production.json

Why?

- Applications behave differently across environments.
- Development may use an in-memory database, while production uses SQL Server.
- Logging levels (detailed in Development, minimal in Production).
- API endpoints differ per environment.

These files allow **override of values** defined in appsettings.json for specific environments, without changing the main configuration file.

Default Structure of AppSettings.json File

By default, a new ASP.NET Core project contains:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Here,

- **Logging** → Defines log levels.
- **AllowedHosts** → Specifies which hosts the app can accept requests from. Here, * means from all hosts, including Swagger, Postman, Fiddler, etc.

Example: Adding Custom Keys in AppSettings.json File

We can add our own configuration values in appsettings.json. For example, please modify the **appsettings.json** file as follows. This allows you to keep application-specific settings structured, reusable, and changeable over time without requiring a redeployment of the application.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MyAppSettings": {
    "ApplicationName": "MyFirstWebAPIProject",
    "Version": "1.0.0",
    "DefaultPageSize": 20
  }
}
```

Real-time Analogy: Think of appsettings.json as a **blueprint or rulebook** for your application. It instructs the system on how to behave in various environments without altering the core logic, thereby making it easier to manage growth and complexity.

Accessing Configuration Information in ASP.NET Core Web API

ASP.NET Core provides the **IConfiguration** interface to access configuration data. This approach is the **same across all configuration sources** (JSON, Environment Variables, Command Line, etc.), because ASP.NET Core merges them into a single unified configuration system.

Creating MyAppSettings

First, create a class file to store the MyAppSettings data. Create a class file named **MyAppSettings.cs** within the **Models** folder and copy and paste the following code. Here, we are giving the property name the same as the key names defined in the appsettings.json file.

```
namespace MyFirstWebAPIProject.Models
{
    public class MyAppSettings
    {
        public string? ApplicationName { get; set; }
        public string? Version { get; set; }
        public int DefaultPageSize { get; set; }
    }
}
```

```
}  
}
```

Creating a Test Controller to Access Configuration

Let's demonstrate how to read from the appsettings.json file. So, please create an Empty API Controller named **ConfigTestController** within the Controllers folder, and copy-paste the following code. In the code below, I demonstrate the various ways to read the configuration data.

```
using Microsoft.AspNetCore.Mvc;  
using MyFirstWebAPIProject.Models;  
  
namespace MyFirstWebAPIProject.Controllers  
{  
    [Route("api/[controller]")]  
    [ApiController]  
    public class ConfigTestController : ControllerBase  
    {  
        private readonly IConfiguration _configuration;  
  
        public ConfigTestController(IConfiguration configuration)  
        {  
            _configuration = configuration;  
        }  
  
        [HttpGet]  
        public IActionResult GetConfigValues()  
        {  
            // Method 1: Direct access using keys  
            var appName = _configuration["MyAppSettings:ApplicationName"];  
  
            // Method 2: Access using GetSection  
            var pageSize =  
_configuration.GetSection("MyAppSettings:DefaultPageSize").Value;  
  
            // Method 3: Binding to a strongly-typed object  
            // Here, the Key Names must be same as the Property Names  
            var settings = new MyAppSettings();  
            _configuration.GetSection("MyAppSettings").Bind(settings);  
  
            return Ok(new  
            {  
                ApplicationName = appName,  
                DefaultPageSize = pageSize,  
                StronglyTyped = settings  
            });  
        }  
    }  
}
```

```
}  
}  
}
```

Testing api/ConfigTest Endpoint:

Now, if you access the **https://localhost:7191/api/ConfigTest** endpoint, then you will see the following response. Here, I am testing using Swagger.

Request URL

https://localhost:7191/api/ConfigTest

Server response

Code	Details
200	Response body <pre>{ "applicationName": "MyFirstWebAPIProject", "defaultPageSize": "20", "stronglyTyped": { "applicationName": "MyFirstWebAPIProject", "version": "1.0.0", "defaultPageSize": 20 } }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 25 Sep 2025 06:16:47 GMT server: Kestrel</pre>

Default Order of Reading Configuration Sources

ASP.NET Core reads configuration sources in a specific order. Later sources override earlier ones:

1. appsettings.json
2. appsettings.{Environment}.json (e.g., Development, Production)
3. User Secrets (in Development)
4. Environment Variables
5. Command-Line Arguments

Suppose we define a key **MyAppSettings:ApplicationName** in different sources:

- In **appsettings.json** → “**MyFirstWebAPIProject**”
- In **appsettings.Development.json** → “**MyFirstWebAPIProject – Dev**”
- In **Command-Line Argument** → **dotnet run MyAppSettings:ApplicationName=“OverriddenByCLI”**

Final Value = **OverriddenByCLI** (because Command-Line Arguments have the highest precedence).

Modifying appsettings.Development.json

Now, please create or modify **appsettings.Development.json** to override some of these values as follows.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "MyAppSettings": {
    "ApplicationName": "MyFirstWebAPIProject - Development",
    "Version": "1.0.1-Dev",
    "DefaultPageSize": 5
  }
}
```

Here:

- ApplicationName changes to indicate this is a **Development Environment**.
- Version shows a dev-specific suffix.
- DefaultPageSize is reduced for testing purposes.

How ASP.NET Core Reads It

- The base file **appsettings.json** is always loaded first.
- Then, **appsettings.Development.json** is loaded if your environment is set to **Development**.
- Keys in the environment file override the base ones.

Therefore, in **the Development environment**, the values above will replace those from **appsettings.json**.

How to Test It

Inside your project's **Properties/launchSettings.json**, make sure the environment variable is set to Development. So, please modify the **launchSettings.json** file as follows.

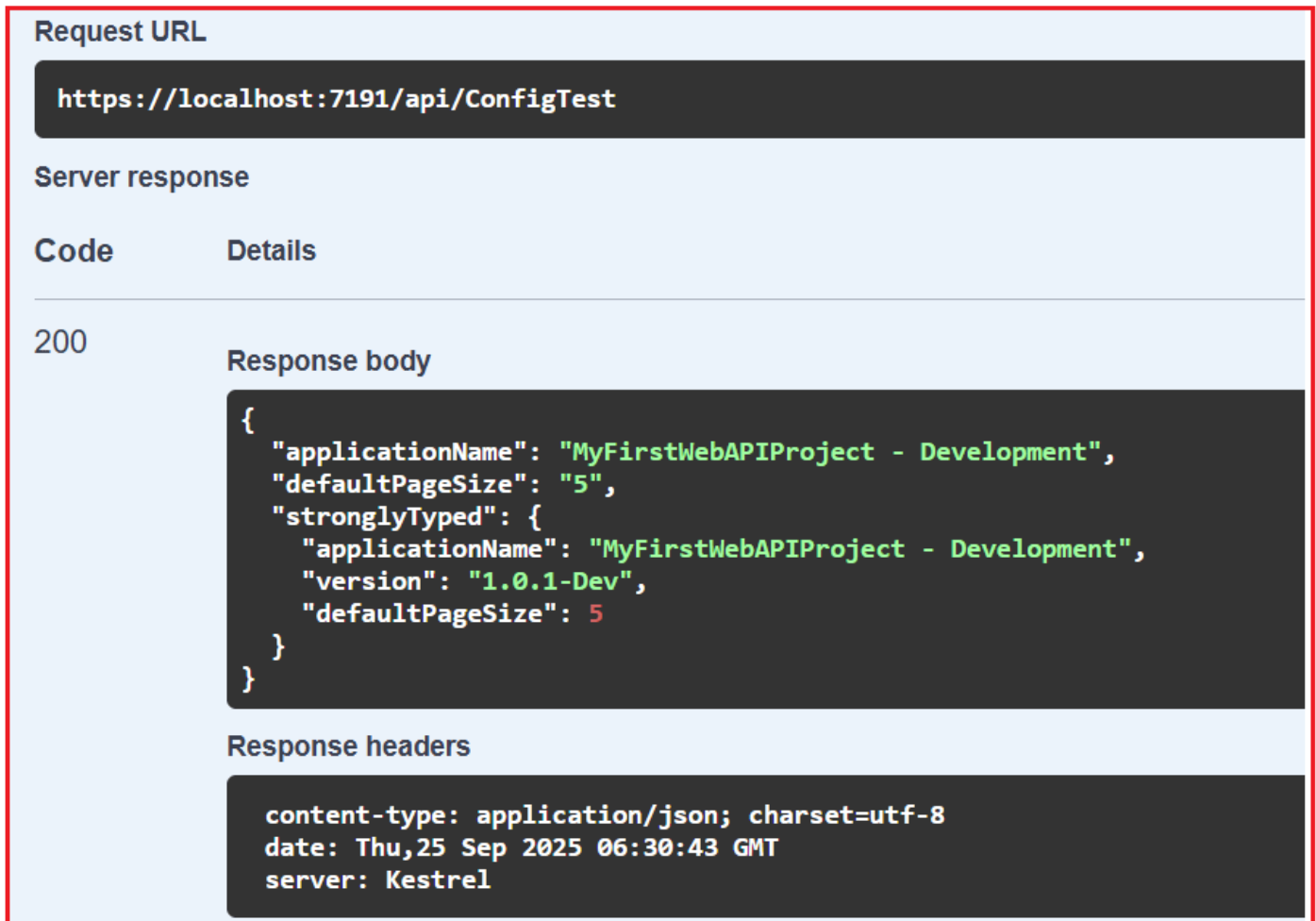
```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:13420",
      "sslPort": 44318
    }
  },

  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5021",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },

    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7191;http://localhost:5021",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },

    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

When you run the project from Visual Studio or use the dotnet run command, the environment is automatically set to Development. With the above changes in place, run the application and test the endpoint using Swagger. You should then see the configuration values loaded from the `appsettings.Development.json` file, as shown in the image below.



Request URL

`https://localhost:7191/api/ConfigTest`

Server response

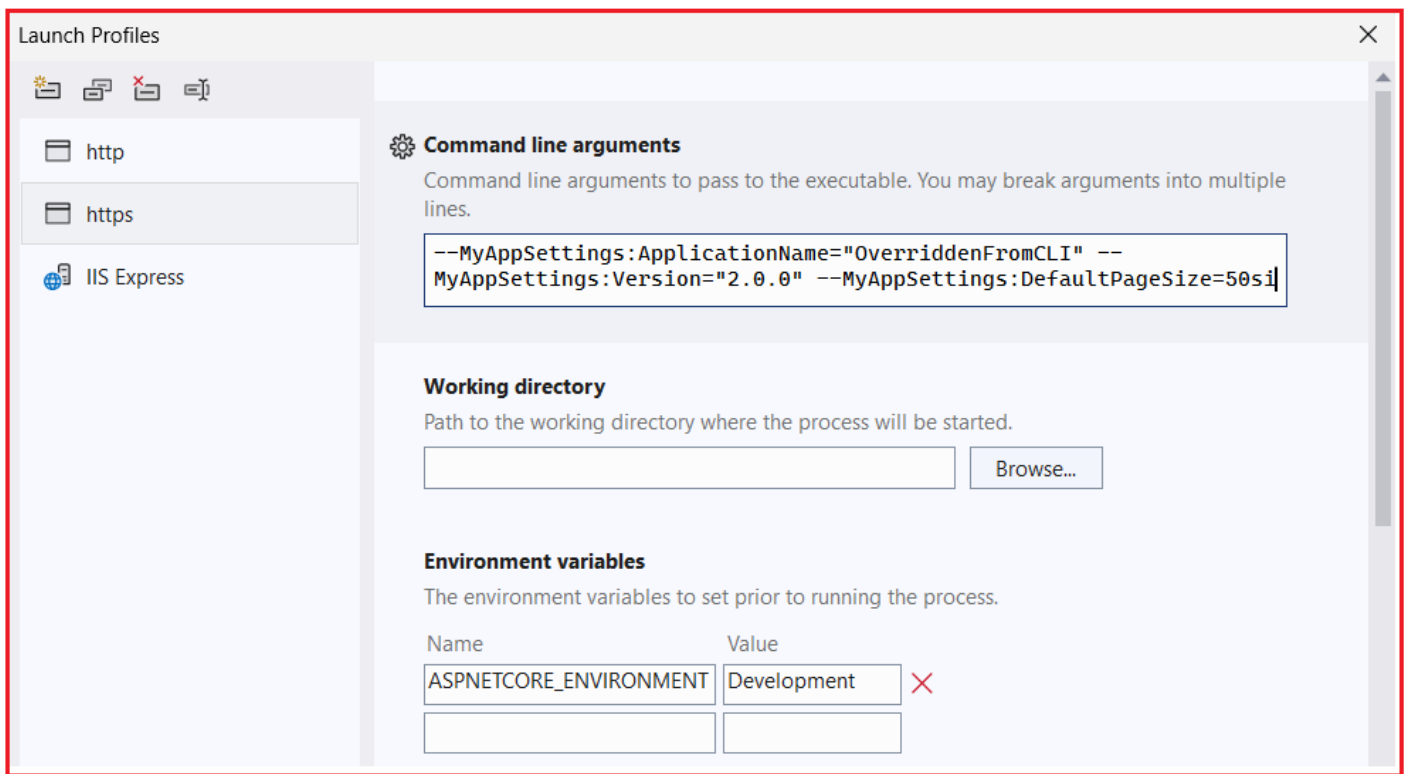
Code	Details
200	Response body <pre>{ "applicationName": "MyFirstWebAPIProject - Development", "defaultPageSize": "5", "stronglyTyped": { "applicationName": "MyFirstWebAPIProject - Development", "version": "1.0.1-Dev", "defaultPageSize": 5 } }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 25 Sep 2025 06:30:43 GMT server: Kestrel</pre>

Using Command Line Argument:

In Visual Studio, we can easily pass command-line arguments to our ASP.NET Core application using the project properties. So, right-click on the project in the Solution Explorer and select Properties from the context menu. This opens the properties window of our project.

In the properties window, go to the Debug tab and then click on the Open Debug Launch Profiles UI. Look for a section named Command line arguments. This field allows us to specify the arguments that the application should receive on startup.

Here, please enter the command line arguments in this field, for example, `-MyAppSettings:ApplicationName="OverriddenFromCLI"` `-MyAppSettings:Version="2.0.0"` `-MyAppSettings:DefaultPageSize=50` as shown in the image below.



Click "Save" on the toolbar or press Ctrl+S to save the settings. As we are setting the command line arguments for the https launch profile, run the application using https, and you should see the configuration values loaded from the **command line**, as shown in the image below.

Request URL

`https://localhost:7191/api/ConfigTest`

Server response

Code

Details

200

Response body

```
{
  "applicationName": "OverriddenFromCLI",
  "defaultPageSize": "50",
  "stronglyTyped": {
    "applicationName": "OverriddenFromCLI",
    "version": "2.0.0",
    "defaultPageSize": 50
  }
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,26 Sep 2025 13:35:37 GMT
server: Kestrel
```

How to Set the Environment in Staging and Production?

When we publish our ASP.NET Core Web API to **Staging** or **Production**, the `launchSettings.json` file is **not included**. The `launchSettings.json` is **only for local development**. In such cases, the environment is set differently depending on where you deploy.

Using Environment Variable (Recommended)

ASP.NET Core looks at the environment variable `ASPNETCORE_ENVIRONMENT` at runtime. Set it to one of the standard values:

- Development
- Staging
- Production

Linux / macOS (bash):

- export ASPNETCORE_ENVIRONMENT=Staging
- dotnet MyFirstWebAPIProject.dll

Windows (PowerShell):

Step 1: Navigate to Your Project Folder

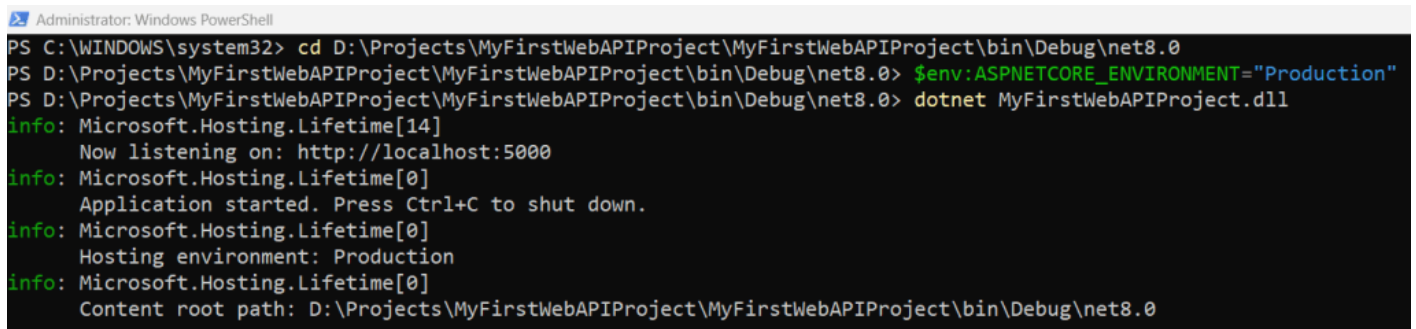
Move into your project's output directory: `cd D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject\bin\Debug\net8.0`

(or if you published it with `dotnet publish`, go to `bin\Release\net8.0\publish`).

Step 2: Run the Application with Environment

- `$env:ASPNETCORE_ENVIRONMENT="Production"`
- `dotnet MyFirstWebAPIProject.dll`

This is the **most common way** when running on servers, Docker, or cloud platforms.



```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> cd D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject\bin\Debug\net8.0
PS D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject\bin\Debug\net8.0> $env:ASPNETCORE_ENVIRONMENT="Production"
PS D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject\bin\Debug\net8.0> dotnet MyFirstWebAPIProject.dll
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Projects\MyFirstWebAPIProject\MyFirstWebAPIProject\bin\Debug\net8.0
```

Using IIS (Windows Hosting)

If hosting in **IIS with the ASP.NET Core Module**:

1. Open your app's folder → `web.config`.
2. Add `<environmentVariables>` section:

```
<configuration>
  <system.webServer>
    <aspNetCore processPath="dotnet" arguments=".\\MyFirstWebAPIProject.dll"
      stdoutLogEnabled="false">
      <environmentVariables>
        <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Staging" />
      </environmentVariables>
    </aspNetCore>
  </system.webServer>
</configuration>
```

Using Azure App Service (Cloud Hosting)

In the Azure Portal → go to your App Service → **Configuration** → **Application Settings**. Add a new setting:

- **Name Value**
- **ASPNETCORE_ENVIRONMENT Production**

Azure automatically injects this setting as an environment variable when the app runs.

Using Docker

If running inside a container, add an environment variable in **Dockerfile** or **docker-compose.yml**:

Dockerfile:

```
ENV ASPNETCORE_ENVIRONMENT=Production
```

docker-compose.yml:

environment:

```
- ASPNETCORE_ENVIRONMENT=Staging
```

What is the Default Lifetime of IConfiguration?

In ASP.NET Core, the default lifetime of **IConfiguration** is **Singleton**. This means the configuration system is built once when the application starts (inside Program.cs via WebApplication.CreateBuilder(args)), and the resulting configuration root is registered in the dependency injection container as a single shared instance.

Because of this, every controller, service, or class that depends on IConfiguration gets the exact same object reference. This design ensures consistency, avoids unnecessary rebuilding of configuration providers, and provides a single source of truth for all application settings.

Key Points to Remember:

- IConfiguration is registered as a **Singleton**.
- Created once at startup (WebApplication.CreateBuilder(args)).
- Shared across the entire application.
- Guarantees consistent configuration values everywhere.
- Prevents costly repeated initialization of providers.

How does IConfiguration handle changes?

Although IConfiguration is a singleton, its values are not frozen. ASP.NET Core's configuration system supports **reload-on-change** for certain providers, such as JSON files, environment variables, and Azure Key Vault.

For example, appsettings.json is added with reloadOnChange: true, which sets up a FileSystemWatcher to monitor the file. If the file changes, the configuration root updates in place. Since all consumers share the same singleton instance, they will see the updated values the next time they access configuration keys. This is why you can modify appsettings.json during development, save it, and immediately see updated values without restarting the application.

Key Points to Remember:

- Singleton instance but supports **dynamic reloads**.
- reloadOnChange: true enables automatic updates.
- Uses **FileSystemWatcher** for JSON files.
- When the appsettings.json file changes, its values are refreshed in memory.
- All consumers see updates because they reference the same instance of the singleton.
- Implemented in framework code (ApplyDefaultAppConfigurationSlim).

If you look up the definition of WebApplicationBuilder, you will see a method called ApplyDefaultAppConfigurationSlim, which contains the configuration.

GitHub	Link	for	Source	Code:
	https://github.com/dotnet/aspnetcore/blob/main/src/DefaultBuilder/src/WebApplicationBuilder.cs			

Options Pattern in ASP.NET Core

The **Options Pattern** is a recommended approach in ASP.NET Core for **reading, organizing, and utilizing configuration data**. Instead of repeatedly fetching configuration values from IConfiguration using string keys, you **bind configuration sections to strongly-typed classes**.

Benefits:

- **Strongly-Typed Access** → IntelliSense & compile-time safety.
- **Centralized Configuration** → Settings are grouped into classes.
- **Easier Maintenance** → Avoids magic strings ("MyAppSettings:ApplicationName") all over your code.
- **Testability** → You can mock or inject different configuration values easily.

Real-time Analogy: Instead of looking up every ingredient separately from a messy cupboard (_configuration["Sugar"], _configuration["Salt"]), you keep them organized in a labeled container (RecipeOptions).

How to Implement the Options Pattern

Let us proceed and understand how to implement the Options pattern in an ASP.NET Core Application.

Step 1: Define a POCO Class for Settings

Create a class that represents a section in appsettings.json. So, create a class file named **MyAppSettingsOptions.cs** within the **Models** folder, and copy-paste the following code.

```
namespace MyFirstWebAPIProject.Models
{
    public class MyAppSettingsOptions
    {
        public string ApplicationName { get; set; } = string.Empty;
        public string Version { get; set; } = string.Empty;
        public int DefaultPageSize { get; set; }
    }
}
```

Step 2: Add Configuration in appsettings.json

We have already added the configuration in the **appsettings.json** file. Please ensure the following:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MyAppSettings": {
    "ApplicationName": "MyFirstWebAPIProject",
    "Version": "1.0.0",
    "DefaultPageSize": 20
  }
}
```

Step 3: Register Options in Program.cs

We need to bind the configuration section to our options class within the program Class file.

```
builder.Services.Configure<MyAppSettingsOptions>
(builder.Configuration.GetSection("MyAppSettings"));
```

Step 4: Use Options in Controller

Inject **IOptions<T>** into your controller. Therefore, please modify the `ConfigOptionsController` as follows to utilize the Options Pattern for reading the configuration.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Options;
using MyFirstWebAPIProject.Models;

namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ConfigTestController : ControllerBase
    {
        // Strongly typed settings class injected via the Options Pattern
        private readonly MyAppSettingsOptions _settings;

        // The constructor receives IOptions<MyAppSettingsOptions> from DI
        // 'options.Value' gives us the bound configuration values
        public ConfigTestController(IOptions<MyAppSettingsOptions> options)
        {
            // Store the configuration values for use in actions
            _settings = options.Value;
        }

        // GET: api/configtest
        [HttpGet]
        public IActionResult GetConfigValues()
        {
            // Returning configuration values as JSON response
            // These values are coming from appsettings.json (or environment-
            // specific overrides)
            return Ok(new
            {
                ApplicationName = _settings.ApplicationName, // e.g.
                "MyFirstWebAPIProject"
                Version = _settings.Version, // e.g. "1.0.0"
                DefaultPageSize = _settings.DefaultPageSize // e.g. 20
            });
        }
    }
}
```

Different Types of Options

ASP.NET Core gives three flavors depending on your needs:

IOptions<T>:

Singleton reads values once at startup. Best for configs that never change while the app is running (e.g., app name, version).

- Loaded **once at application startup**.
- Same values are returned for the lifetime of the application.
- Best for **static configs** (AppName, Version).

IOptionsSnapshot<T>:

Scoped, refreshed on every request (only in scoped lifetime). Best for web apps where configs may change between requests.

- Values are **refreshed on every request**.
- Useful for **web apps** where config may change between requests (but not mid-request).
- Only works with **Scoped lifetime** (per request).

IOptionsMonitor<T>:

Singleton supports change notifications. Best if configs may change at runtime (e.g., reloading values without restart).

- Always available as a **Singleton**.
- Can notify when settings **change at runtime** (with file reloads or external providers).
- Best for scenarios like **dynamic logging levels, feature toggles, or live updates**.

Example to Understand the different Options used:

Please modify the ConfigOptionsController as follows:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Options;
using MyFirstWebAPIProject.Models;

namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ConfigTestController : ControllerBase
    {
```



```

private readonly MyAppSettingsOptions _settingsFromOptions;
private readonly MyAppSettingsOptions _settingsFromSnapshot;
private readonly IOptionsMonitor<MyAppSettingsOptions> _settingsMonitor;

// Constructor injects all three types
public ConfigTestController(
    IOptions<MyAppSettingsOptions> options,
    IOptionsSnapshot<MyAppSettingsOptions> snapshot,
    IOptionsMonitor<MyAppSettingsOptions> monitor)
{
    // IOptions<T> → Singleton-like, values fixed at app startup
    _settingsFromOptions = options.Value;

    // IOptionsSnapshot<T> → Scoped, refreshed per request (only in Web
apps)
    _settingsFromSnapshot = snapshot.Value;

    // IOptionsMonitor<T> → Singleton, supports change notifications and
real-time reload
    _settingsMonitor = monitor;
}

[HttpGet("compare")]
public IActionResult CompareOptions()
{
    return Ok(new
    {
        // These three may return different values if config changes
during runtime
        FromIOptions = new
        {
            _settingsFromOptions.ApplicationName,
            _settingsFromOptions.Version,
            _settingsFromOptions.DefaultPageSize
        },
        FromIOptionsSnapshot = new
        {
            _settingsFromSnapshot.ApplicationName,
            _settingsFromSnapshot.Version,
            _settingsFromSnapshot.DefaultPageSize
        },
        FromIOptionsMonitor = new
        {
            _settingsMonitor.CurrentValue.ApplicationName,
            _settingsMonitor.CurrentValue.Version,
            _settingsMonitor.CurrentValue.DefaultPageSize
        }
    });
}

```

```
    });  
  }  
}
```

What happens if `appsettings.json` (or another config source) changes while the app is running? Will `IConfiguration`, `IOptions<T>`, `IOptionsSnapshot<T>`, and `IOptionsMonitor<T>` automatically reflect the new value?

`IConfiguration`

- **Lifetime: Singleton**
- **Yes, it updates live** (if `reloadOnChange: true`).
- It uses a `FileSystemWatcher` (for JSON) or equivalent provider hook.
- The next time you access `_config["MyKey"]`, you'll get the updated value.
- Because it's a **singleton**, all consumers see the same updated data.

`IOptions<T>`

- **Lifetime: Singleton** (bound once at startup and cached).
- **No, it does not update.**
- Values are captured once at **application startup**.
- Even if `appsettings.json` changes, the injected `IOptions<T>.Value` will keep the **old values** until you restart the app.
- Suitable for static configurations, such as app name and version.

`IOptionsSnapshot<T>`

- **Lifetime: Scoped** (new instance per request in web apps).
- **Yes, but only per request (Scoped).**
- On every new HTTP request, ASP.NET Core rebinds the config section → so the new request will see updated values if the file has changed.
- But during the middle of an ongoing request, values don't change.
- Works only in **web apps** (requires scoped lifetime).

`IOptionsMonitor<T>`

- **Lifetime: Singleton** (one instance across the app, but internally refreshes values).
- **Yes, live updates in real-time.**
- When config changes, the monitor refreshes the values **immediately**.
- Best option when you need **runtime reloading** without restarting the app.

The **AppSettings.json** file in ASP.NET Core Web API provides a centralized and flexible way to manage configuration data. With support for environment-specific overrides, multiple configuration sources, and strong integration with the IConfiguration interface, it ensures that your application remains adaptable across different environments. By understanding how configuration is structured, accessed, and overridden, developers can build APIs that are both maintainable and production-ready.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career.

Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information